
PROBLEM SOLVING AGENTS: SEARCHING Cont.

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

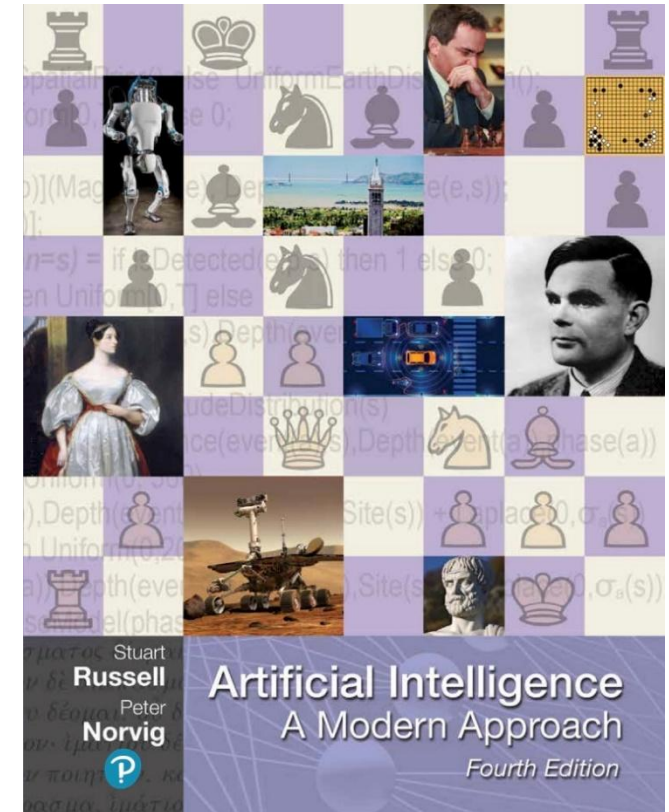
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 3.5, 3.6 in AIMI 4/E



Outline

- Informed Search Algorithms:
 - Use of heuristics to guide the search
 - Greedy best-first search
 - A* search
- Formulating heuristics
- Summary

Informed (Heuristic) Search Algorithms

Search Algorithms [1]

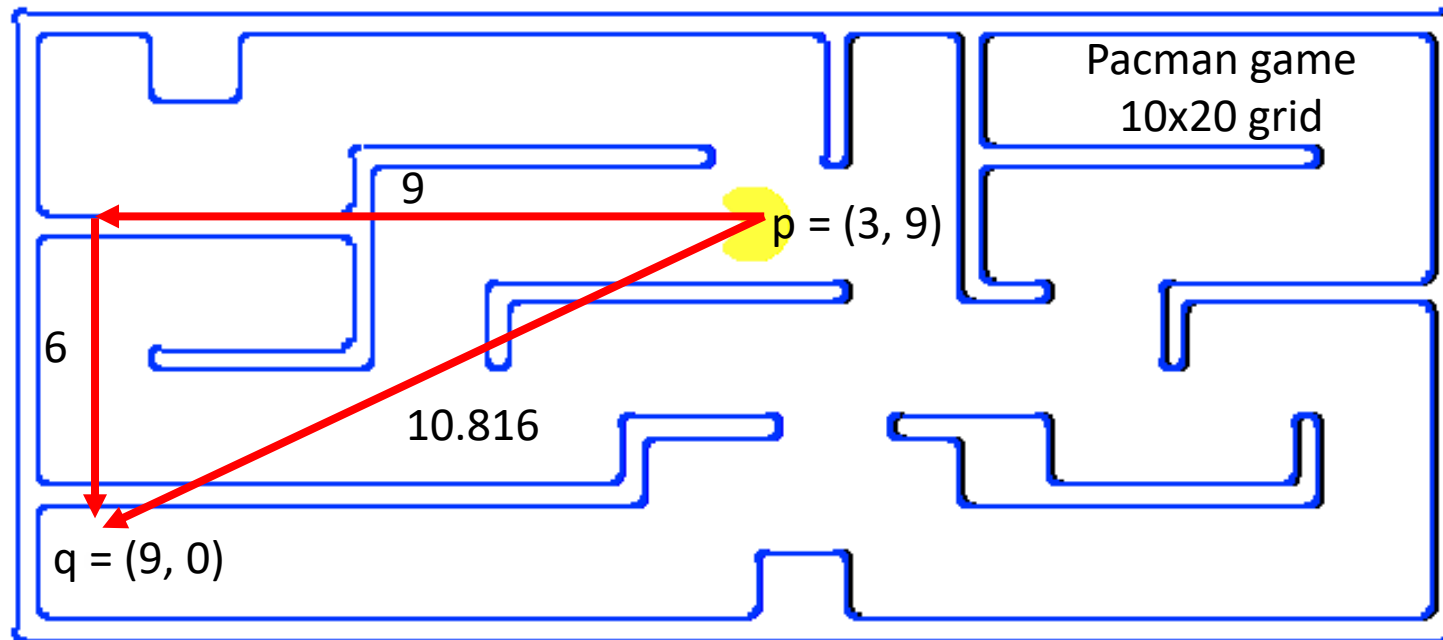
- How to change the behavior of a search algorithm using the best-first search template? We just need to use different evaluation functions $f(n)$.
- We discuss two categories of search algorithms.
- **Uninformed** search algorithms: no information about states beyond what is given in the problem. definition.
- **Informed (heuristic)** search: makes use of a **heuristic function** to guide the search.
 - Greedy search
 - A* search

Search Algorithms [2]

- **Uninformed (blind) search:**
 - Time and space demanding for real complex problems.
 - Can only recognize goal once it's achieved.
 - Hence, no notion of “right direction”.
- **Informed (heuristic) search:**
 - Uses a heuristic to evaluate how good various states are to guide the search.
 - With good heuristic, can reduce the search significantly

Search Algorithms – Heuristics

- **Informed (heuristic)** search: makes use of a heuristic function to guide the search.
- **Heuristic function $h(n)$: *estimated*** cost of the cheapest path from the state at node n to a goal state.
- Thus, a heuristic function that better approximates the true cost will reduce the amount of searching.
- Typically, designed for a particular search problem (problem specific).
- Example: Manhattan distance, Euclidean distance, etc.



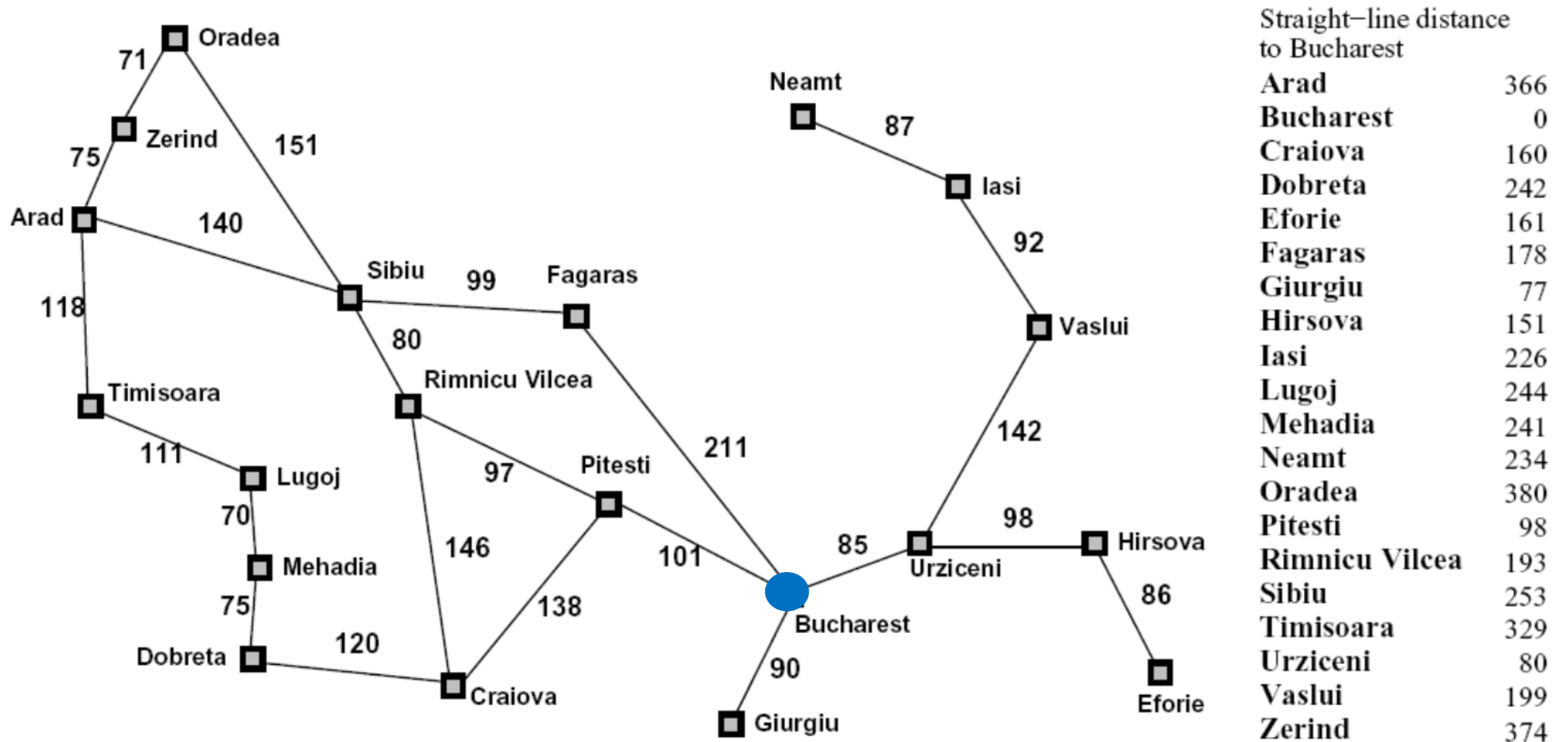
Given the n -dimensional points p and q (vectors), we can calculate distance as:

$$\text{manhattandist}(p, q) = \sum_{i=1}^n |p_i - q_i|$$

$$\text{euclideanist}(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Example: Travelling in Romania – straight-line heuristic

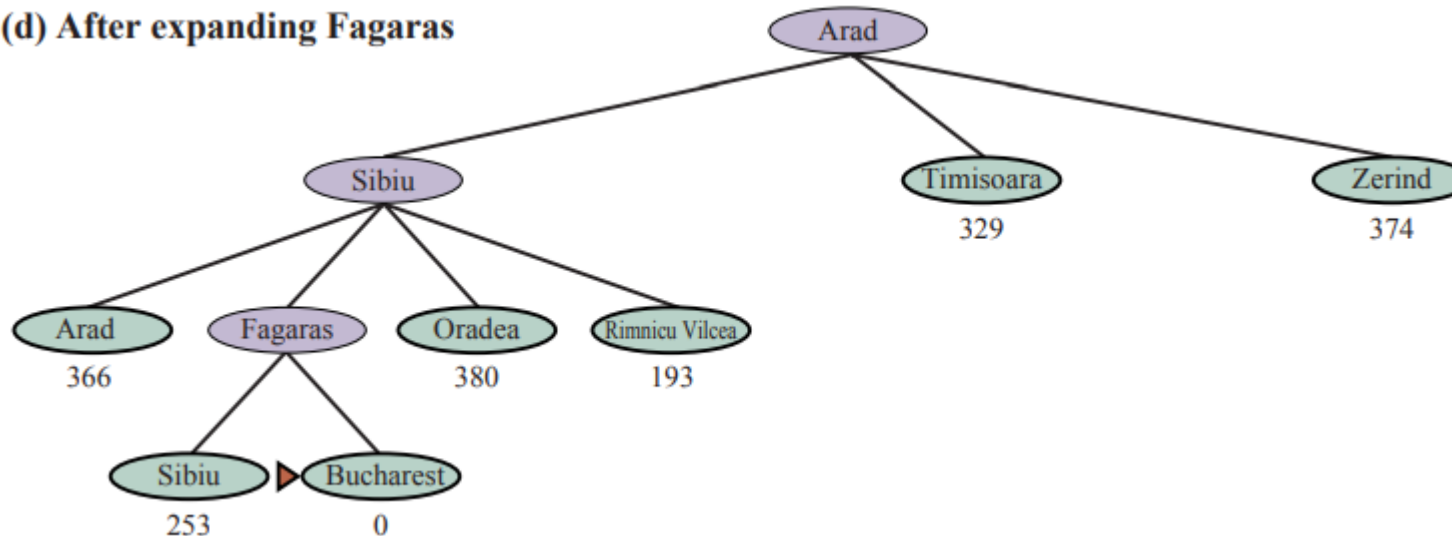
- In route-finding problems like ‘travelling in Romania’, a reasonable heuristic is straight-line distance from a state to the goal state. Also known as Euclidean distance. (assume you are given 2D coordinates of cities)



Greedy search [1]

- Uses the best-first search template with a heuristic function. That is, $f(n) = h(n)$.
- Thus, it selects the node from the frontier with **minimum $h(n)$** , i.e. the node that appears nearest to the goal.
- Using straight-line distance on Romania problem, we will get the solution Arad -> Sibiu -> Fagaras -> Bucharest.
- Better focused search with heuristics. However, it is not optimal path.

(d) After expanding Fagaras

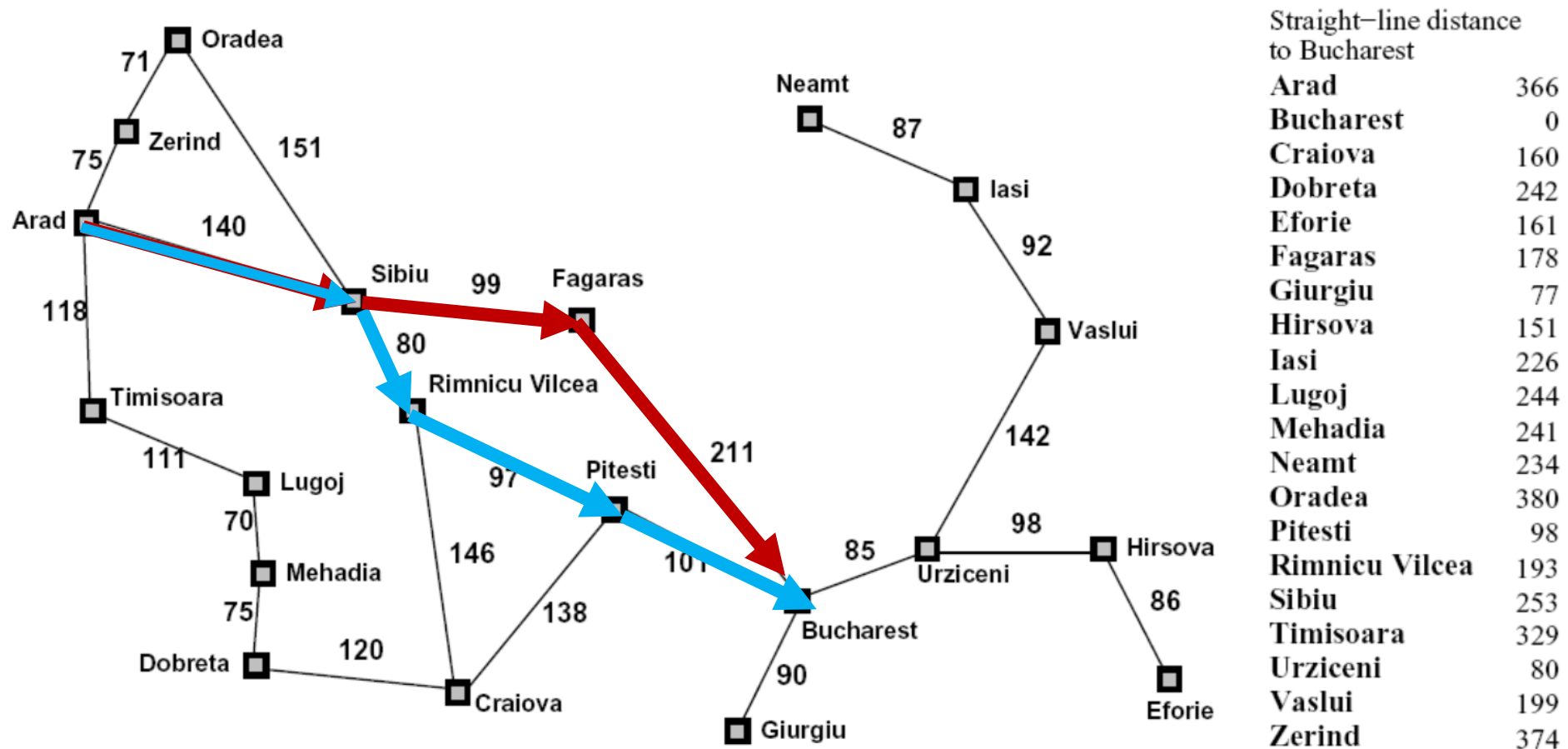


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy search [2]

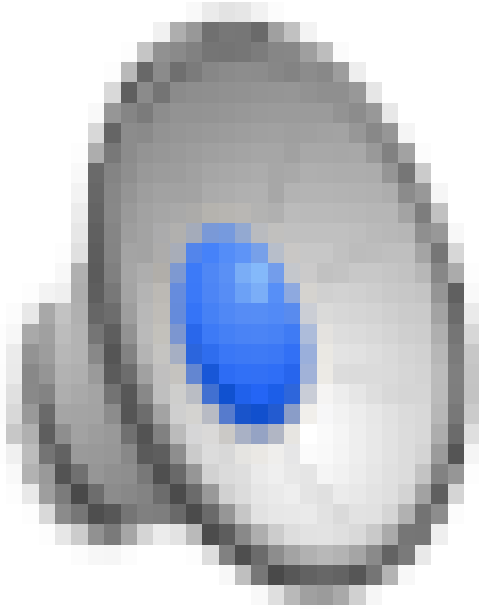
- Using straight-line distance on Romania problem, **greedy search** returns the solution Arad -> Sibiu -> Fagaras -> Bucharest. This has cost 450.
- Better focused search with heuristics. However, it is not **optimal path** (shown below in blue with cost 418).



Greedy search [3]

- Complete?
 - Tree-like: Depends on the heuristic used. If state-space is infinite or has cycles, you could make up a heuristic that sets you down an infinite path and never to the goal. Complete in finite state spaces without cycles (with no cycles, you will eventually get to goal).
 - Graph-like: complete in finite state spaces with or without cycles (at some point you will hit the goal). Incomplete in infinite state spaces with odd behaving heuristic.
- Optimal?
 - No, greediness in the heuristic can lead to suboptimal results.
- Worst-case space and time complexity?
 - Tree-like: $O(b^m)$ in finite state spaces without cycles.
 - Graph-like: $O(b^m)$ in finite state spaces with or without cycles.

Greedy search behavior with Euclidean distance heuristic
Action costs are 1. Gives suboptimal solution.



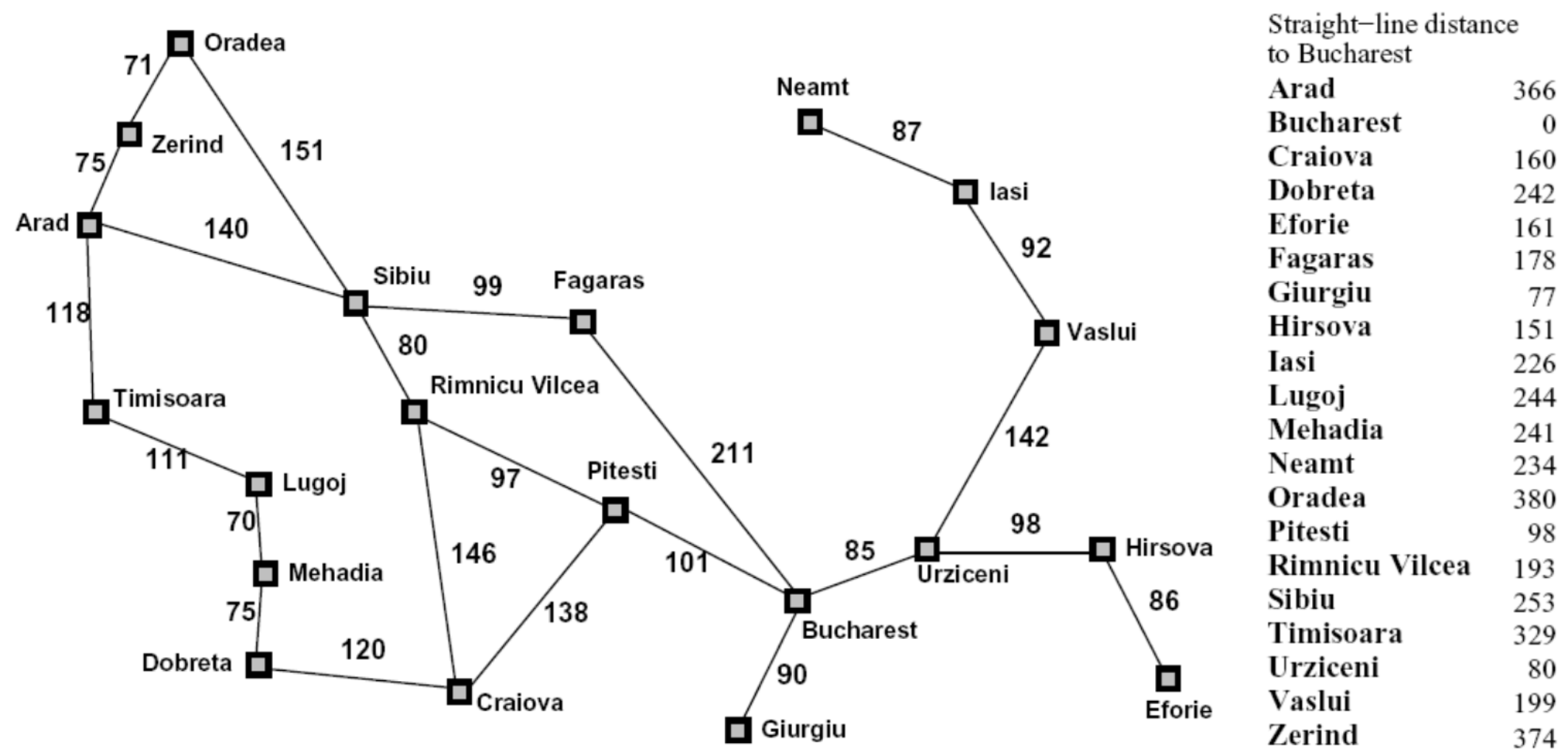
Video from [link](#)

A* search

- UCS orders states by path cost $g(n)$
- $g(n)$ is the **actual** path-cost from the initial state to the state at node n .
- Greedy orders states by some defined heuristic $h(n)$
- $h(n)$ is the heuristic function that **estimates** the cheapest cost from the state in node n to a goal state.
- A* combines UCS and greedy $f(n) = g(n) + h(n)$
- Thus, $f(n)$ is the estimated cost of best path that continues from state in node n to a goal.
- The better $h(n)$ approximates the true cost to the goal, the more efficient A* will be in its search.
- By efficiency we mean it generates fewer nodes in the search tree.

A* search example: Travelling Romania [1]

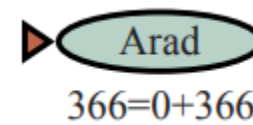
- The actual road costs are shown on the state-space graph. The straight-line heuristic is shown in the table.



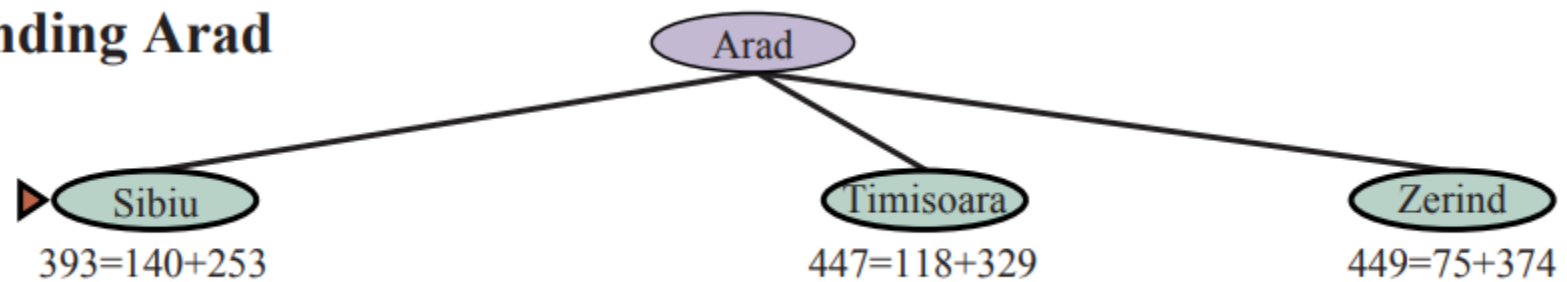
A* search example: Travelling Romania [2]

- A* progression on travelling Romania problem.

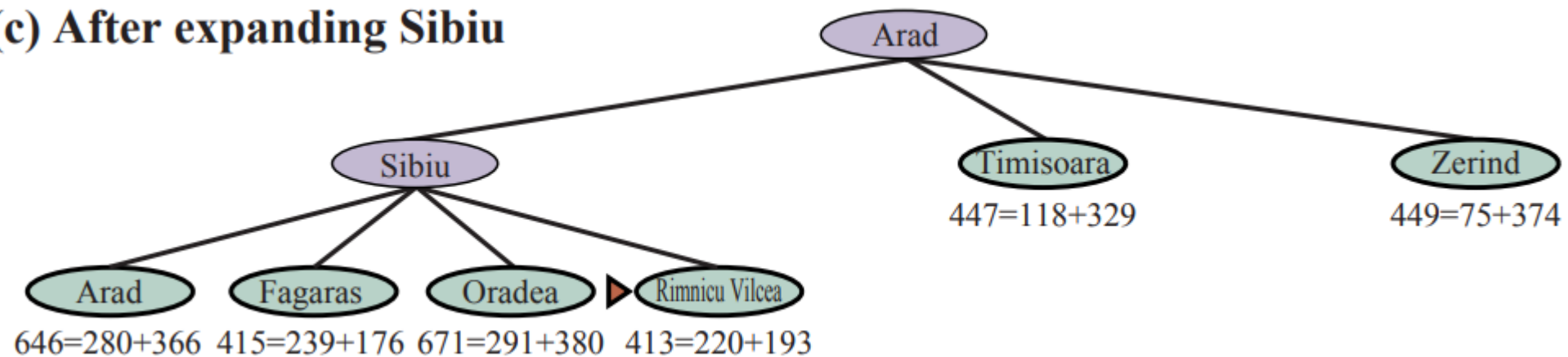
(a) The initial state



(b) After expanding Arad

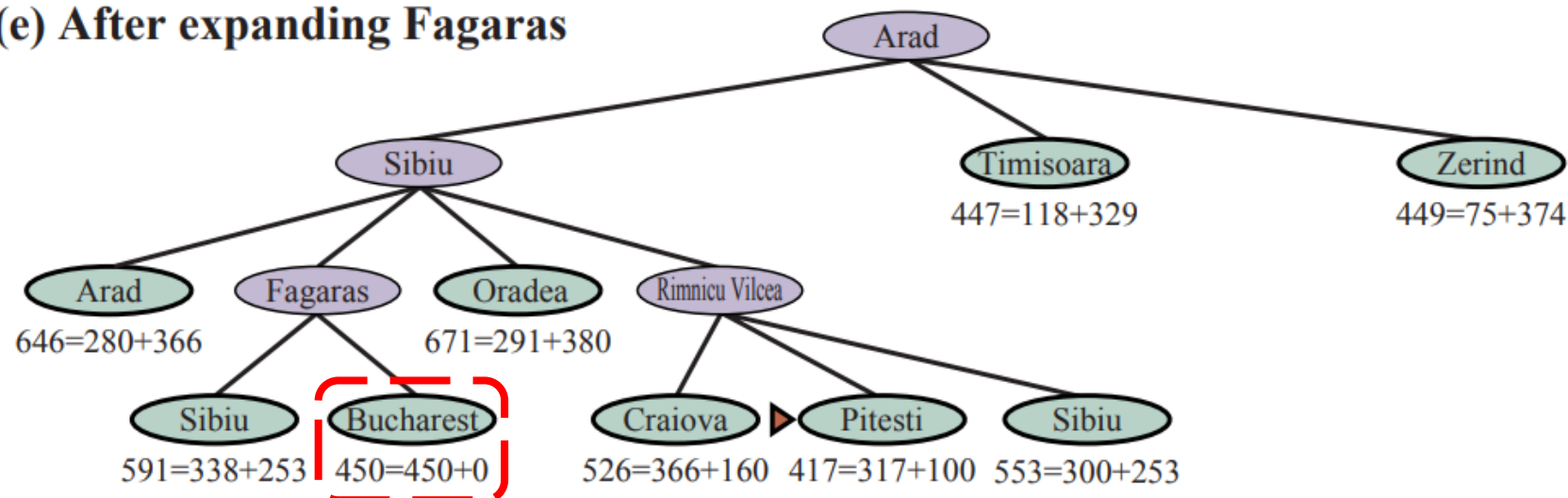


(c) After expanding Sibiu

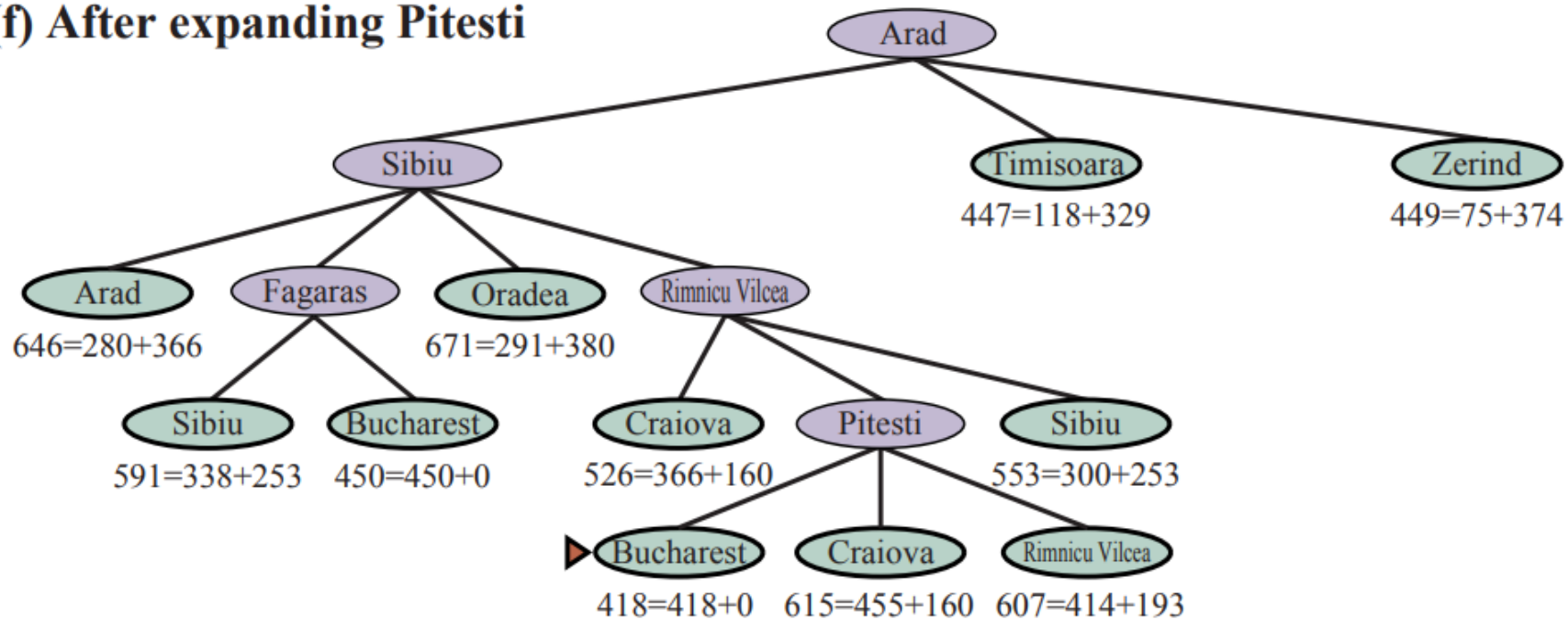


- Note that Bucharest (goal) appears on the frontier early, but it is not 'popped' because it is not the lowest $f(n)$.
- The best-first search template does a **late-goal test**; it tests for goals after popping a node. This is required for A* to be optimal.

(e) After expanding Fagaras

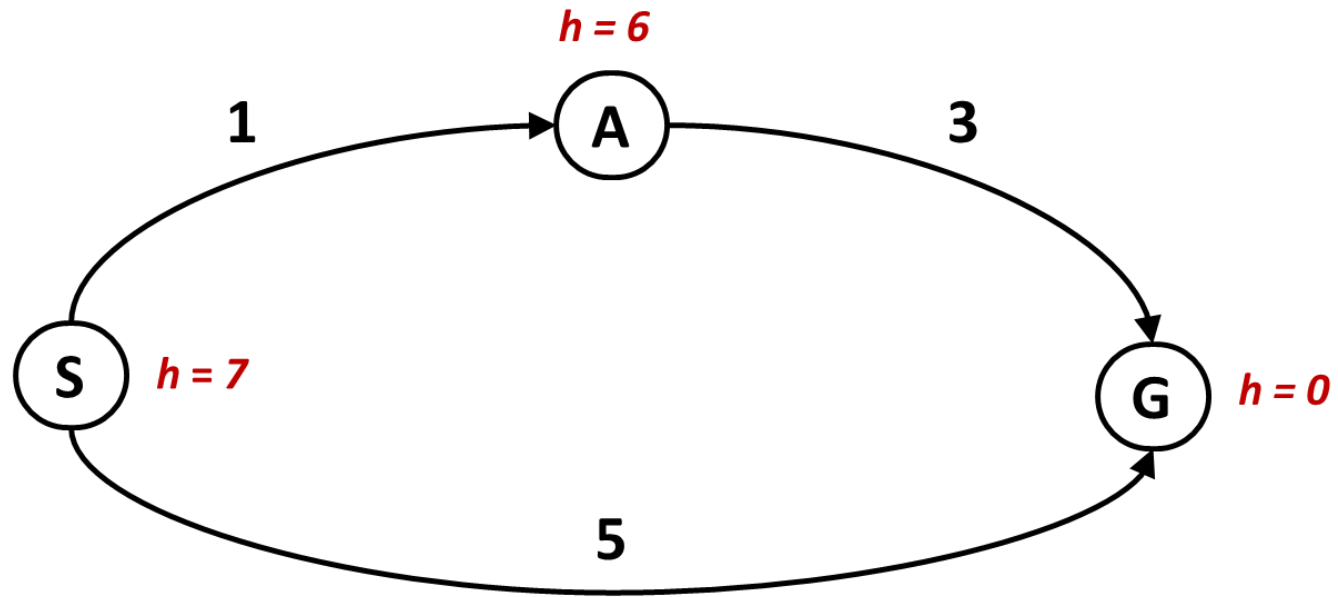


(f) After expanding Pitesti



A* search properties

- A* is complete if action costs are positive, and state space has a solution or is finite.
- Is A* optimal? Depends on the properties of the heuristic. Consider the below problem.



Frontier:

$$A: f(A) = 1 + 6 = 7$$

$$G: f(G) = 5 + 0 = 5$$

Thus, select G for expansion.

- The optimal solution is $S \rightarrow A \rightarrow G$ with cost 4, but using A* with the given heuristic returns the suboptimal solution $S \rightarrow G$ with cost 5. Why?
- Because the heuristic here *overestimates* the *actual* cheapest cost to reach the goal.

A* Optimality [1]

- An **admissible** heuristic *never overestimates* the cost to reach the goal (optimism).
- Mathematically, if $h^*(n)$ is cost of optimal path from state n to goal, then an admissible heuristic $h(n)$:

$$\forall n. 0 \leq h(n) \leq h^*(n)$$

- Note that we usually do not know $h^*(n)$; *otherwise we would have just used $h^*(n)$ as our heuristic.*
- A* tree-like search is optimal if $h(n)$ is **admissible**
- What about A* graph-like search, is it optimal?

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

add *child* to *frontier*

return *failure*

function EXPAND(*problem*, *node*) **yields** nodes

s $\leftarrow$ *node*.STATE

for each *action* **in** *problem*.ACTIONS(*s*) **do**

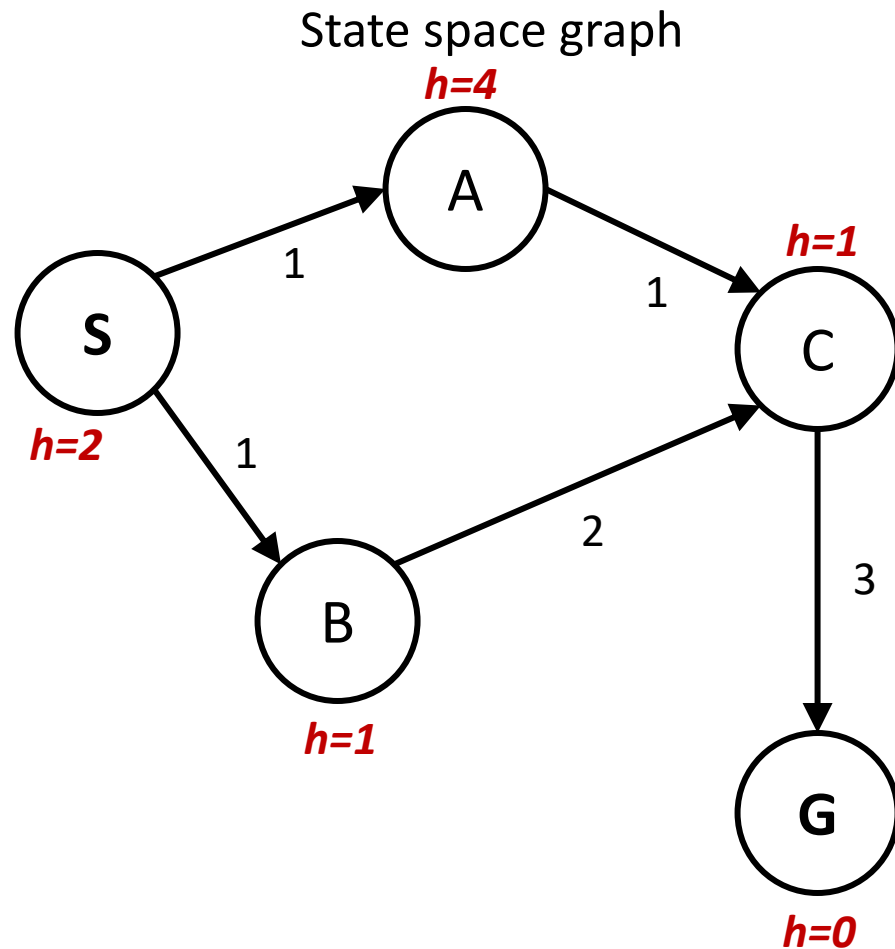
s' $\leftarrow$ *problem*.RESULT(*s*, *action*)

cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)

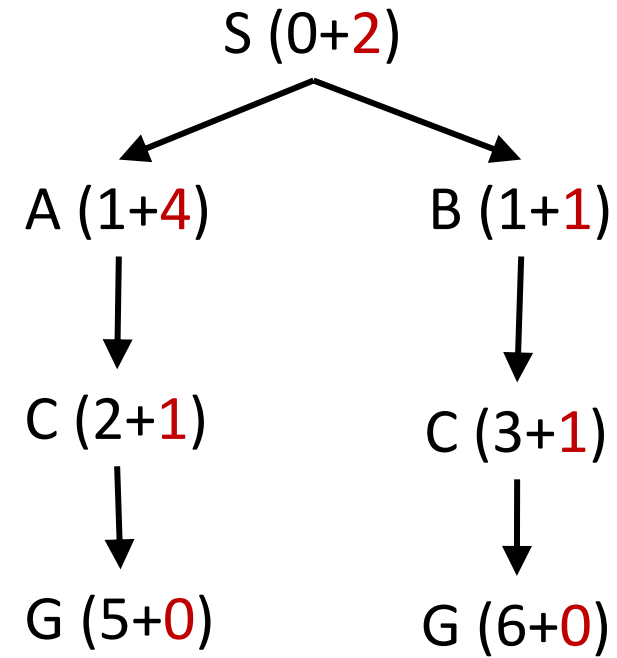
yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

- We mentioned that A* tree-like search using an admissible heuristic is optimal.
- What about A* graph-like search, is it optimal?
- We check for reached states and lower cost paths.
- This lower-cost check is necessary for graph-like A* to be optimal.

A* graph-like optimality – checking reached states



Search tree



C has already been reached C(3+1). But because we found a better path C(2+1), we add that node to the frontier.

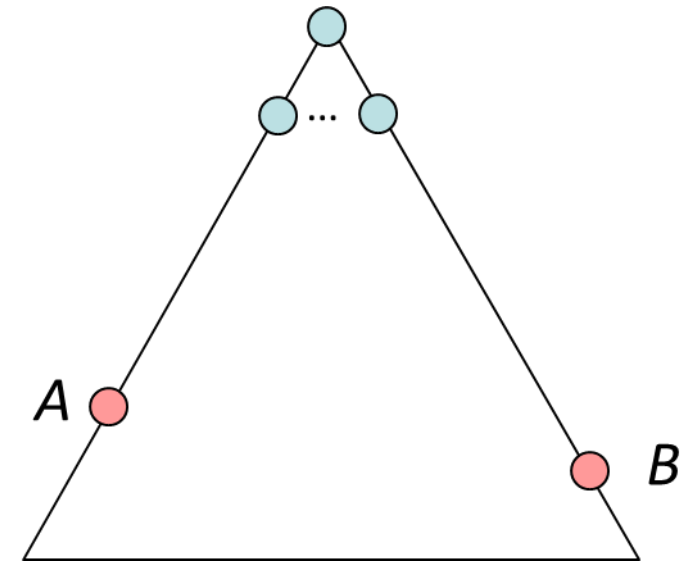
- If we did not add a lower cost node to frontier, then we would have returned suboptimal path (S-B-C-G).
- Thus, A* with the graph-like best-first search template using an admissible heuristic is optimal.

A* Optimality [1]

- A* tree-like search is optimal if h is admissible. A* graph-like best-first search template using an admissible heuristic is optimal.
- An **admissible** heuristic *never overestimates* the cost to reach the goal (optimism).
- Mathematically, if $h^*(n)$ is cost of optimal path from state n to goal, then an admissible heuristic $h(n)$:

$$\forall n. 0 \leq h(n) \leq h^*(n)$$

- Proof for optimality of A*, assume:
 - A is optimal goal node
 - B is suboptimal goal node
 - h is admissible heuristic
- We need to argue that if we encountered a situation where a suboptimal node B is on the frontier *before* an optimal node A, then A* will expand A before B.
- **Claim:** A will be selected (popped) for expansion before B



A* Optimality [2]

- Proof for optimality of A*, assume:

- A is optimal goal node
- B is suboptimal goal node
- h is admissible heuristic

- Claim: A will be selected (popped) for expansion before B

- Proof:

- Suppose B is on the frontier.
- Some node n that is on optimal path (ancestor of A) is also on the frontier.
- Claim: n will be expanded before B

1. $f(n) \leq f(A)$

$$f(n) = g(n) + h(n)$$

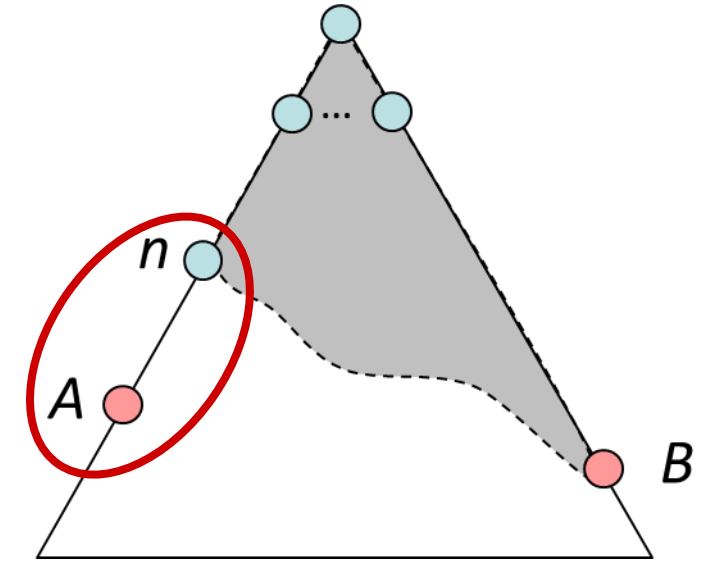
$$\leq g(n) + h^*(n)$$

$$= g(A) = f(A)$$

Definition of f -cost

Admissibility of h : $h(n) \leq h^*(n)$

since n on optimal path and $h(A) = 0$



A* Optimality [3]

- Proof for optimality of A*, assume:
 - A is optimal goal node
 - B is suboptimal goal node
 - h is admissible heuristic

• Claim: A will be selected for expansion before B

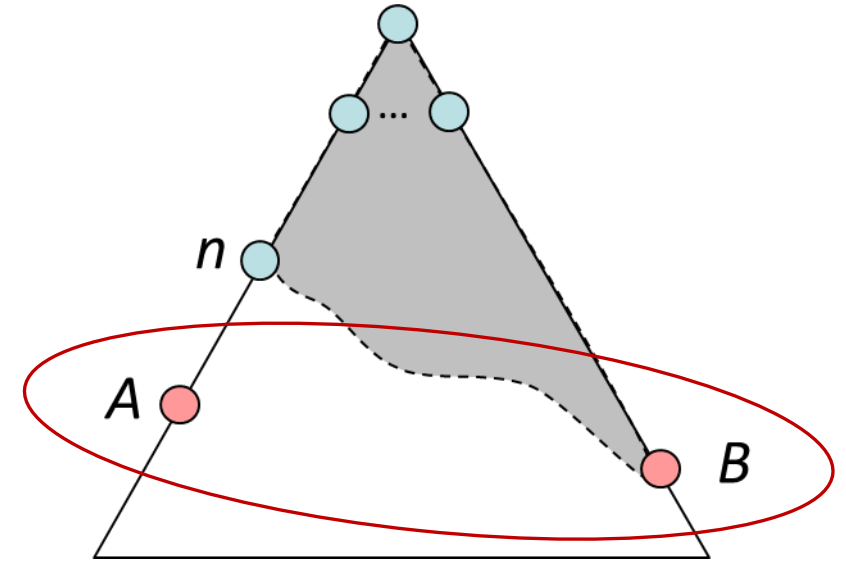
• Proof:

- Suppose B is on the frontier.
- Some node n that is on optimal path (ancestor of A) is also on the frontier.
- Claim: n will be expanded before B

1. $f(n) \leq f(A)$

2. $f(A) < f(B)$

3. From 1 and 2, $f(n) \leq f(A) < f(B)$. That is n is expanded before B



$$g(A) < g(B)$$

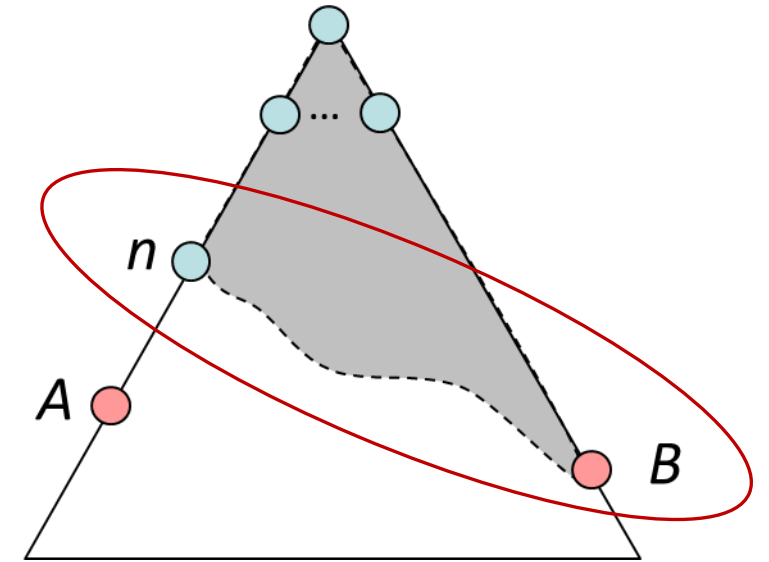
$$f(A) < f(B)$$

Suboptimality of B

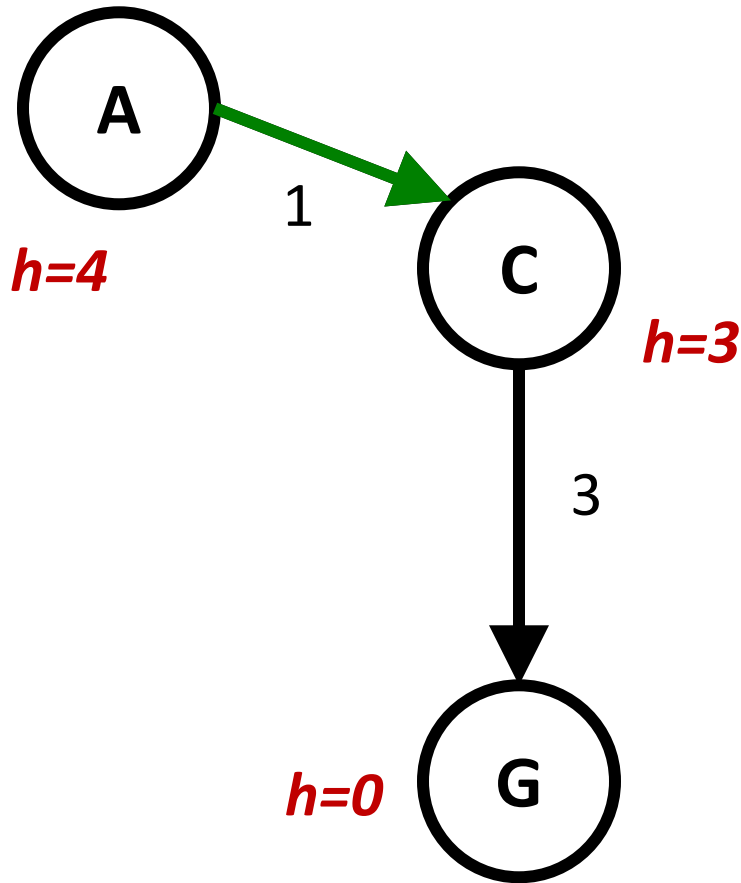
$h = 0$ at a goal

A* Optimality [4]

- Proof for optimality of A*, assume:
 - A is optimal goal node
 - B is suboptimal goal node
 - h is admissible heuristic
- Claim: A will be selected for expansion before B
- Proof:
 - Suppose B is on the frontier.
 - Some n that is an ancestor of A is also on the frontier.
 - Claim: n will be expanded before B
 1. $f(n) \leq f(A)$
 2. $f(A) < f(B)$
 3. From 1 and 2, we get $f(n) \leq f(A) < f(B)$. That is n is expanded before B
- Thus, **ALL** ancestors of A are expanded before B. Because as we've shown, they have $f(n) < f(B)$.
- Also, A is expanded before suboptimal B. Because $f(A) < f(B)$
- Thus, A* is optimal.



Stronger guarantees – Consistent Heuristics



- Admissibility: heuristic estimated path-cost $\leq$ actual cheapest path-cost to goal

$$h(s) \leq h^*(s)$$

- Consistency: heuristic estimated **action-cost** $\leq$ actual **action-cost**
- To show consistency, you need to show that for **all transitions (edges)** in state-space graph between two states s and s' , we have:

$$h(s) - h(s') \leq c(s, s')$$

heuristic estimated **action-cost** $\leq$ actual **action-cost**

Consequences of consistency:

- The f value along a path never decreases (monotonicity):

$$h(s) \leq c(s, s') + h(s') \Rightarrow g(s) + h(s) \leq g(s) + c(s, s') + h(s')$$

$$\Rightarrow f(s) \leq g(s') + h(s') = f(s')$$

- The first time we **pop** a state it will be on an optimal path for that state (cheapest path from initial state to *this* state).

Recap of Admissible and Consistent

- Let $h^*(n)$ be cost of optimal path from state n to goal.
- $h(n)$ is **admissible** if:

$$\forall n. 0 \leq h(n) \leq h^*(n)$$

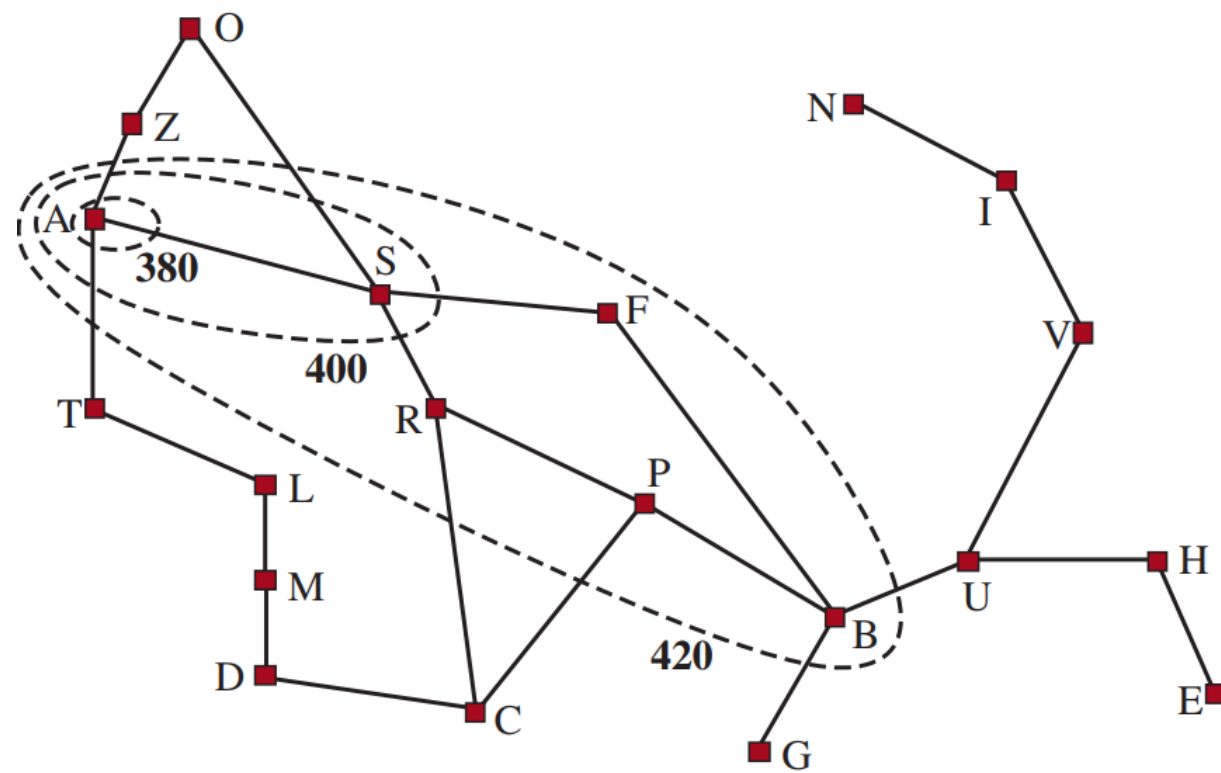
- $h(n)$ is **consistent** if for every state n and every successor state n' of n generated by any action a :

$$\forall n \forall n'. h(n) - h(n') \leq c(n, a, n')$$

- Note that every consistent heuristic is admissible; opposite not necessarily true.
- Consistent heuristics that are good estimates will do less search work, then non-consistent heuristics.
- Given a state-space with action costs, and heuristic values. How can you tell if heuristic is admissible?
How can you tell if heuristic is consistent?

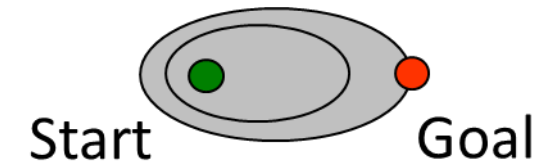
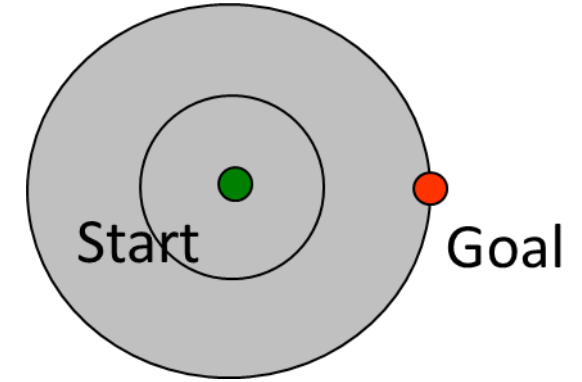
A* Cost Contours

- A* expands nodes in order of increasing f value.
- This defines cost '**contours**' on the state-space graph.
- Contour i has all nodes with $f(n)$ less than some contour value f_i .
- With uniform-cost search ($h(n) = 0$), contours will be circular as nodes are expanded.
- With A* search, contours will expand towards the goal state. With good heuristics, contours will stretch towards optimal path.
- A* will expand all nodes with cost $f(n) < C^*$ (optimal path cost).
- In essence, contours visualizes how the search algorithm expands towards the goal in the state space graph.

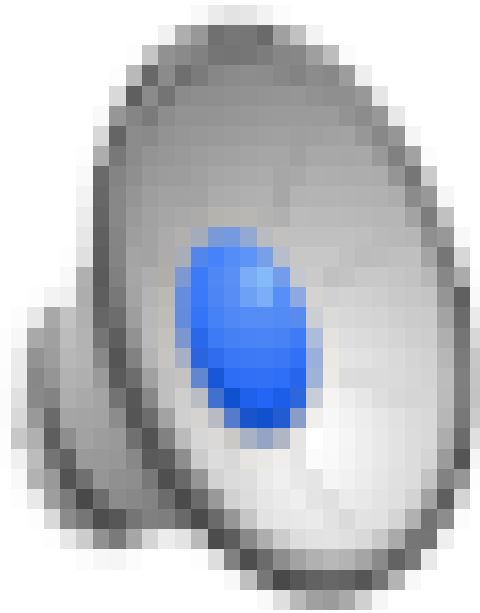


Uniform-cost search vs A* search

- Uniform-cost expands equally in all “directions”
- A* expands mainly toward the goal, with admissible heuristic it ensures optimality.



A* search behavior in grid environment



Video from [link](#)

Greedy vs UCS vs A*



Greedy (h)



Uniform Cost (g)



A* (g+h)

- Notice that UCS searches in a circular manner.
- A* is a mix of UCS and greedy.

Properties of A*

- Complete?
 - Yes, if action costs are positive, and state space has a solution or is finite.
- Optimal?
 - A* tree-like search with admissible heuristic is optimal.
 - A* graph-like best-first search template using admissible heuristic is optimal.
- A* expansion behavior:
 - Cannot expand contour f_{i+1} until contour f_i is explored where $f_i < f_{i+1}$
 - A* expands all nodes with $f(n) < C^*$
 - A* expands some nodes with $f(n) = C^*$
 - A* expands no nodes with $f(n) > C^*$. Reduces search complexity but depends on the quality of the heuristic. Better heuristic gives us more reduction.
- Space and Time?
 - Assuming admissible heuristic, then we can say worst-case is that of UCS with the 0-heuristic. With admissible heuristic better than 0-heuristic, A* does less work than UCS. What about the work with perfect heuristic?
 - What about inadmissible heuristic? Can give odd behavior depending on problem setting. Not guaranteed to be optimal, and may do less or more work than UCS.

How to invent admissible heuristics?

- One simple and effective way to invent an admissible heuristic is to consider a *relaxed* formulation of the given problem.
- A relaxed problem can be formulated by allowing for less restrictions on the actions. E.g. allow an agent to make 3-moves as one action with less cost. (how does this alter the state space graph?)
- Because of the added allowable actions, the state-space graph of the relaxed problem will have more edges between states than the original problem.
- Any optimal solution to the original problem, is also a solution in the relaxed problem. But maybe suboptimal for the relaxed problem.
- Thus, the actual cost of an optimal solution in a relaxed problem is an admissible heuristic in the original problem.
- Note this means we must calculate (compute) the heuristic value from any node n to the goal node in the relaxed problem. So, the heuristic should be fast to compute.

Example: Sliding block puzzles [1]

- States?
 - Any configuration of the tiles. How many possible?
- Actions?
 - Move the *blank* tile to an adjacent location. Left, right, up, down.
- Initial state?
 - Some configuration is set as initial.
- Goal test?
 - Check if current configuration (state) is the desired one.
- Action cost?
 - 1 per move.
- Optimal solution?
 - Get from initial state to goal state in as few costs as possible.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Example: Relaxing sliding block puzzles [2]

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- If we allow a tile to move anywhere. h_1 = number of misplaced tiles
- If we allow a tile to move to any adjacent square. h_2 = sum of distances of tiles from their goal (Manhattan distance)
- $h_1(\text{Start}) = 8$; $h_2(\text{Start}) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$
- Is h_1 admissible? Yes, because any misplaced tile would require at least 1 move.
- Is h_2 admissible? Yes, because any misplaced tile would need to move at least the Manhattan distance to its goal position.

- Implementing h1 and h2. Added as part of the SlidingBlockProblem class.

```
"""
h1 heuristic from lecture slides.
Allow a tile to move anywhere.
h1 = number of misplaced tiles
"""

def h1(self, node):
    curr_state = node.state
    if self.is_goal(curr_state):
        return 0
    else:
        # using numpy features
        unequal_tiles = (curr_state != self.goal_state) # NOTE we are looking at node.state
        num_misplaced_tiles = unequal_tiles.sum()

        return num_misplaced_tiles

"""
h2 heuristic from lecture slides.
Allow a tile to move to any adjacent square.
h2 = sum of distances of tiles from their goal (Manhattan distance)
"""

def h2(self, node):
    curr_state = node.state
    if self.is_goal(curr_state):
        return 0
    else:
        sum_dist_from_goal = 0

        for row in range(3):
            for col in range(3):
                if (self.goal_state[row,col] != curr_state[row,col]): # tile not at goal position
                    misplaced_row, misplaced_col = np.where(curr_state == self.goal_state[row,col])
                    misplaced_row = misplaced_row[0]
                    misplaced_col = misplaced_col[0]

                    manhattan_dist = abs(row - misplaced_row) + abs(col - misplaced_col)
                    sum_dist_from_goal += manhattan_dist

        return sum_dist_from_goal
```

Algorithm	Evaluation function $f(n)$
BFS	depth of node
UCS	$g(n)$ Actual path-cost from start to state at node n
DFS	– (depth of node)
Depth-limited DFS	– (depth of node)
IDS-DFS	– (depth of node)
Greedy	$h(n)$ heuristic cost of state at node n
A^*	$h(n) + g(n)$

Search Algorithms f(n) Summary

- Below is example of `astar_search` that calls `best_first_search()` with correct arguments.

```
def astar_search(problem, h):  
    astar_f_func = (lambda node: node.path_cost + h(node))  
    return best_first_search(problem, f=astar_f_func)
```

Example: Sliding block puzzles [4]

- Comparison between BFS, and A* with h_1 and h_2 . Solution is found at depth d of the search tree.
- h_2 is better than h_1 . It generates fewer nodes.
- We say that h_2 dominates h_1 .

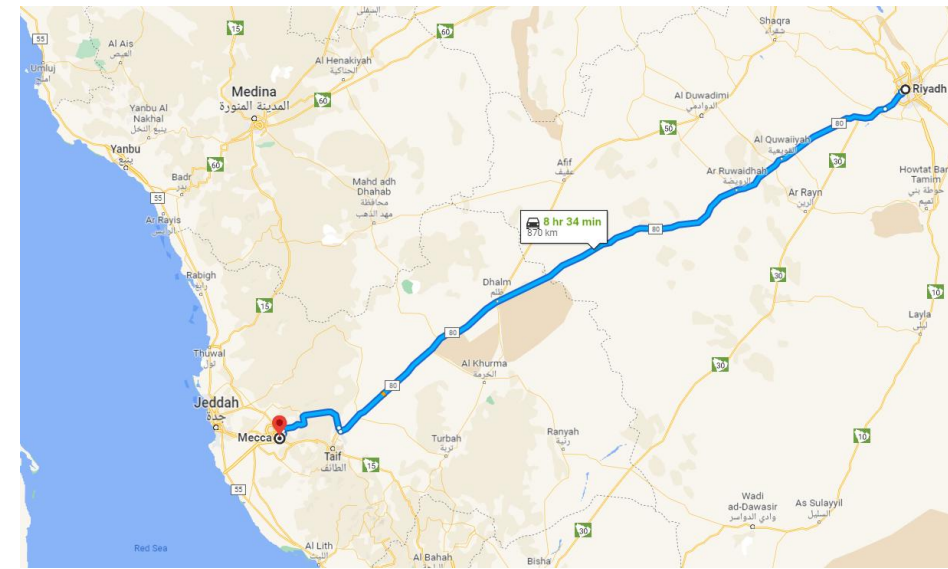
Search Cost (nodes generated)			
d	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19
8	368	48	31
10	1033	116	48
12	2672	279	84
14	6783	678	174
16	17270	1683	364
18	41558	4102	751
20	91493	9905	1318
22	175921	22955	2548
24	290082	53039	5733
26	395355	110372	10080
28	463234	202565	22055

Combining heuristics

- Dominance: Let h_1 and h_2 be admissible heuristics, then h_1 dominates h_2 if: $\forall n. h_1(n) \geq h_2(n)$
 - Roughly speaking, larger heuristic is better as long as both are admissible. Since it approximates the true cost better.
 - The zero heuristic is not informative (what is behavior of A* with $h(n) = 0$?).
 - The exact heuristic is pretty good, but usually too expensive to compute. Heuristics should be fast to compute.
- What if we have m admissible heuristics, none dominates the others?
 - Form a new heuristic by taking the max:
$$h(n) = \max(h_1(n), h_2(n), \dots, h_m(n))$$
 - Max of admissible heuristics is admissible and dominates its components.

A* Applications

- **Pathing / routing problems**
- Resource planning problems
- Robot motion planning
- Video games
- Protein design
- Chemical synthesis



Informed Search Practice

for each *child* **in** EXPAND(*problem*, *node*) **do**

$s \leftarrow \text{child.STATE}$

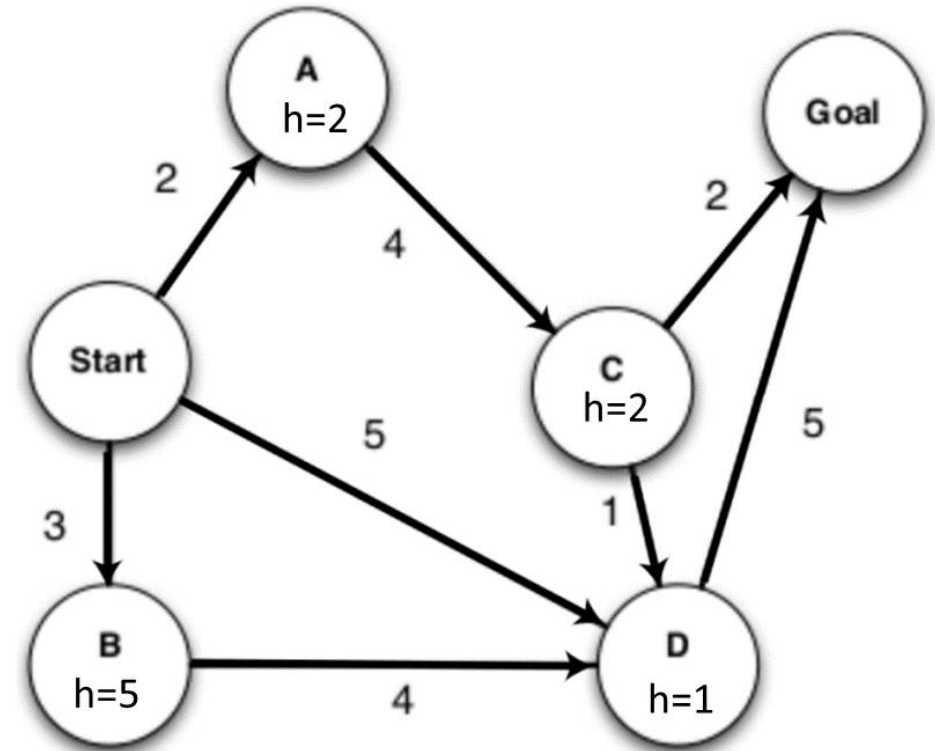
if *s* is not in *reached* **or** *child.PATH-COST* < *reached*[*s*].*PATH-COST* **then**

$\text{reached}[s] \leftarrow \text{child}$

add *child* to *frontier*

- Apply Greedy and A*. Use **graph-like best-first search template**. Assume ties to be resolved in alphabetical order.

- a) list the order in which states are popped from frontier
- b) solution path returned by each strategy and its cost to the goal.
- c) Total number of nodes popped from frontier.
- d) Total number of nodes generated.

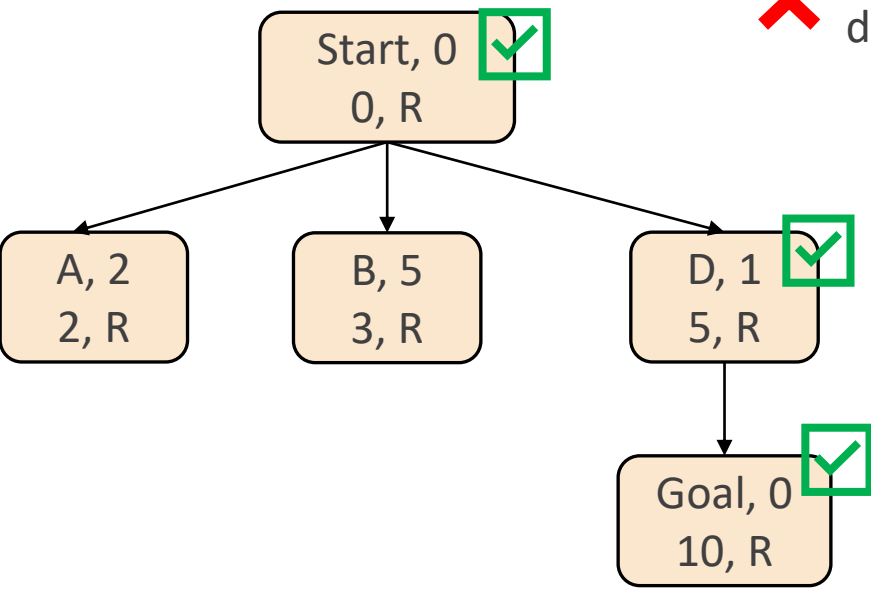


Informed Search Practice – Greedy

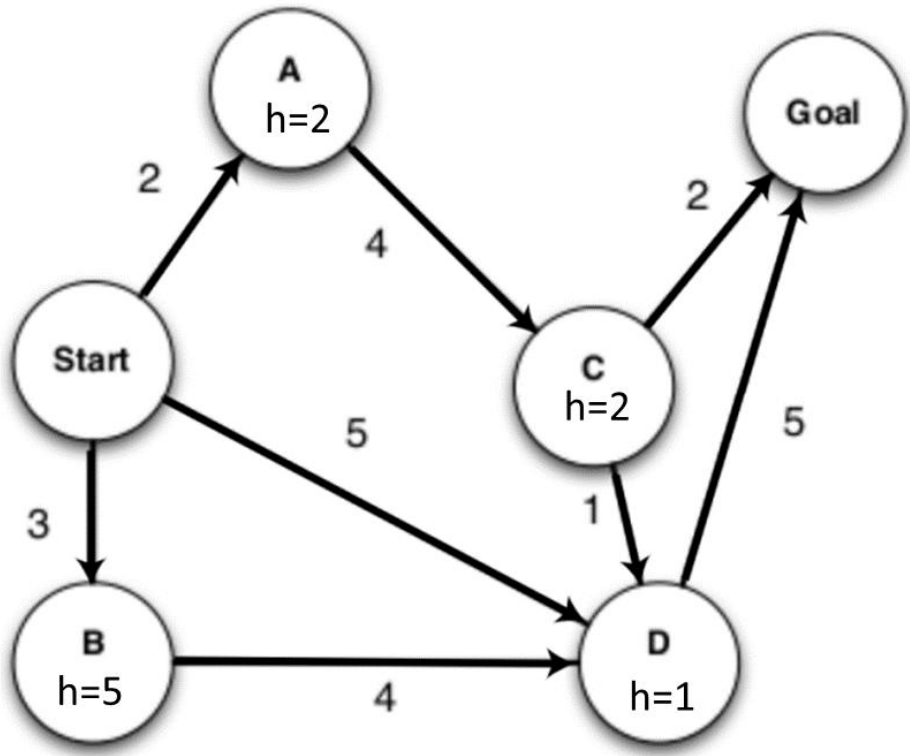
```
for each child in EXPAND(problem, node) do
  s ← child.STATE
  if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
    reached[s] ← child
    add child to frontier
```

State, f-value
path-cost, Reached-pointer

- ✓ Denotes popped node
- ✗ Denotes node generated but discarded (not added to frontier)



Iter#	Popped
1	Start
2	D
3	Goal



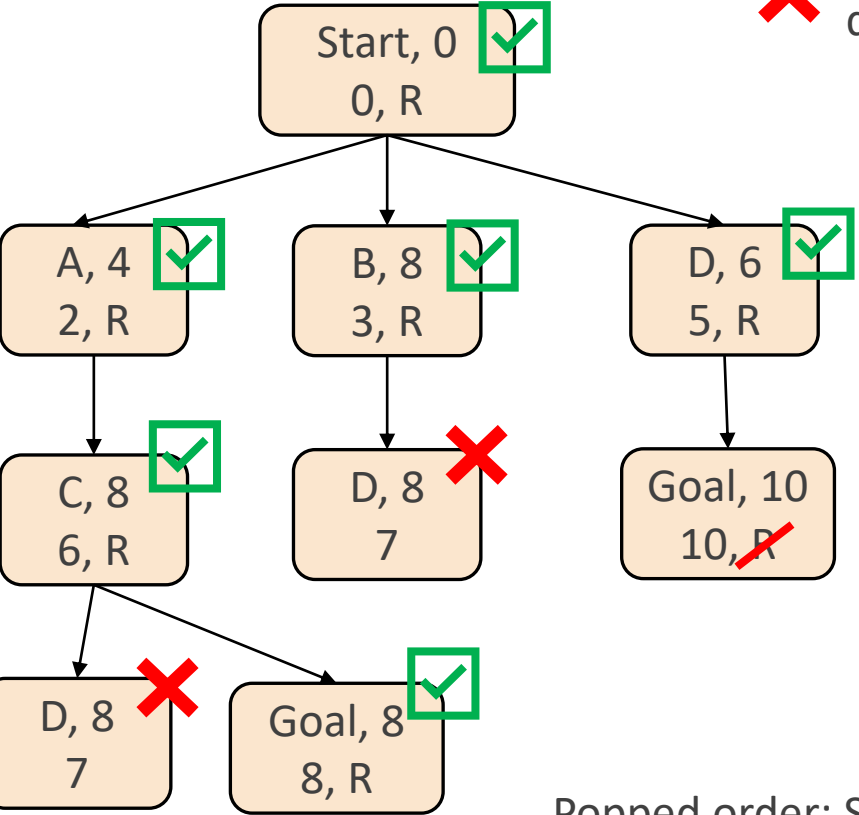
Popped order: Start, D, Goal
Greedy solution path: Start -> D -> Goal
Solution cost: 10
Total nodes popped: 3
Total nodes generated: 5

Informed Search Practice – A*

```
for each child in EXPAND(problem, node) do
  s ← child.STATE
  if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
    reached[s] ← child
    add child to frontier
```

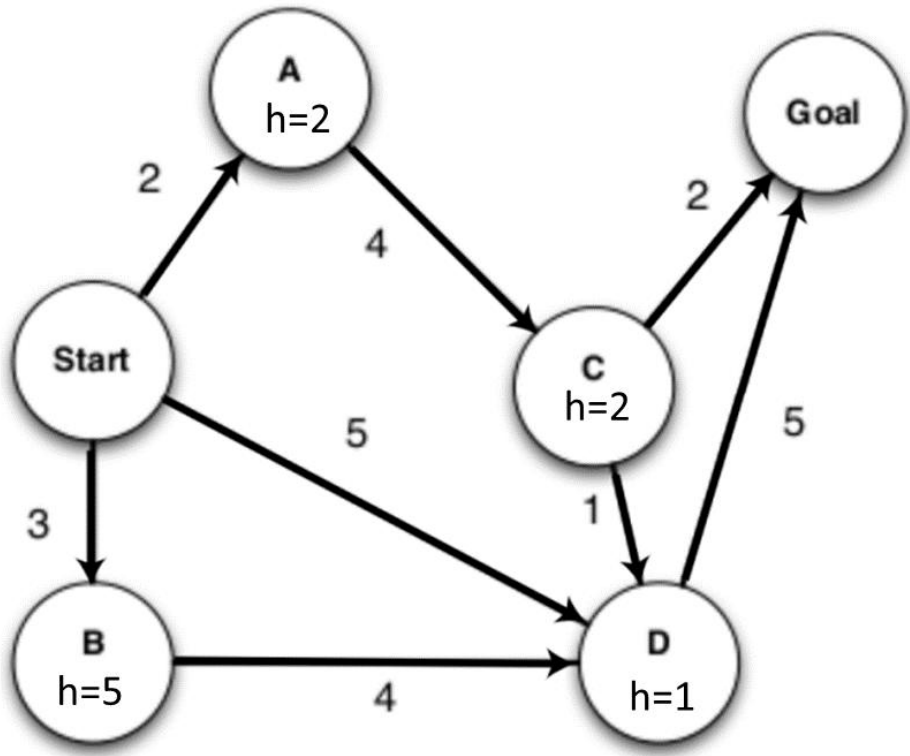
State, f-value
path-cost, Reached-pointer

✓ Denotes popped node
✗ Denotes node generated but discarded (not added to frontier)



Iter#	Popped
1	Start
2	A
3	D
4	B
5	C
6	Goal

Popped order: Start, A, D, B, C, Goal
A* solution path: Start -> A -> C -> Goal
Solution cost: 8
Total nodes popped: 6 Total nodes generated: 9



Summary

Summary

- The search strategy operates over a model of the world,
 - i.e . the agent does not actually try the plans in the real world
 - searching is all “simulation” hence is only as good as the model
- Heuristic functions estimate costs of shortest path to the goal.
- Good heuristics can dramatically reduce search cost.
- Greedy best first search expands lowest h ; not always optimal.
- A* search expands lowest $g + h$; tree-like search complete and optimal with admissible heuristic; graph-like best-first search template complete and optimal using admissible heuristic.
- Admissible heuristics can be derived from exact solution of relaxed problems.
- Every consistent heuristic is admissible (but not vice versa).
- A max of admissible heuristics is also admissible, and dominates its components.